

μOS++ IIIe Tests / xPack System

- The μOS++ IIIe Tests / xPack System
 - 1. Purpose of this document
 - 2. Executive summary
 - 3. Big picture
 - 4. Repository layout
 - 5. The xPack / xpm layer
 - 6. The build-configuration matrix
 - 7. The CMake build system
 - 8. Platforms in detail
 - 9. Toolchains and reproducibility
 - 10. The test suites
 - 11. Running the tests
 - 12. How to make a test
 - 13. How to add a test
 - 14. Extending platforms, toolchains and coverage
 - 15. Chaining and inheritance rules (package.json and CMakeLists.txt)
 - 16. Known limitations and caveats
 - 17. File reference map
 - 18. Glossary

The μOS++ IIIe Tests / xPack System

A structured, reproducible and extensible test harness for @micro-os-plus/micro-os-plus-iii, built on top of **xpm**, **xPacks**, **CMake** and **CTest**, that compiles and runs the *same* test sources on many toolchains, on many emulated and physical platforms.

1. Purpose of this document

This document explains how the test infrastructure of the micro-os-plus-iii project works end to end:

- the **xPack/xpm** package and build model that drives it,
- the **build configurations** that enumerate the platform × toolchain × build-type matrix,
- the **CMake** target graph that actually compiles and links the tests,
- the **platforms** (native, QEMU, physical boards) and how tests are launched on each of them,
- how the whole thing stays **reproducible**, and
- how to **extend** it (new tests, new platforms, new toolchains).

It is intended for maintainers, contributors and anyone who wants to add coverage without breaking the existing structure.

2. Executive summary

The strategy is simple to state and non-trivial to implement:

Write the tests once, then compile and run them with as many toolchains as possible, on as many platforms as possible.

Concretely:

- The test *sources* live in `tests/sources/<name>` and are **platform agnostic**. The exact same files are used everywhere, without changes.
- Each *platform* (`tests/platforms/<name>`) provides only the glue needed to turn those sources into an executable and to run it: a platform library, a device package, linker scripts, compile/link options, and the command used by CTest to launch the result.
- Each *toolchain* is selected by a CMake **toolchain file** provided by the shared `@micro-os-plus/build-helper xPack`.
- Each **(platform, toolchain, build type)** combination is declared as an xpm **build configuration** in `tests/package.json`. Toolchain binaries (`arm-none-eabi-gcc`, `qemu-arm`, `openocd`, `gcc`, `clang`, ...) are declared as xPack **devDependencies** and installed locally under `tests/xpacks/...`, so the build does not depend on globally installed tools.
- The matrix is driven from the shell with `xpm run <action> --config <name>` and from CI with a small set of aggregate actions.

The result is a system where a single `xpm run test-all` can exercise the RTOS APIs under GCC and clang on the host, and under `arm-none-eabi-gcc` on four QEMU Cortex-M variants and several physical boards — all from the same source tree and all with pinned tool versions.

3. Big picture

The system has five layers. Each layer only depends on the one below it.

Layout

5. CI / human entry points	
xpm run test / test-all / test-ci / test-cortex-cmake ...	
.github/workflows/ci.yml	
4. xpm build configurations (tests/package.json → buildConfigurations)	
native-cmake-gcc14-debug, qemu-cortex-m7f-cmake-gcc-release, ...	
each = platform × toolchain × build type, with inherited actions	
3. CMake build system (tests/CMakeLists.txt, tests/cmake/*)	
interface libraries + add_test() + CTest	
2. Sources & platforms	
tests/sources/<test>	tests/platforms/<platform>
tests/device-qemu-cortexm	toolchain files from build-helper
1. xPack / xpm package model	
package.json, tests/xpacks/, .bin, devDependencies (pinned tools)	

The key idea: **layer 2 (the test sources) is invariant**; layers 3–5 select *how* and *where* it is compiled and run.

4. Repository layout

Layout

```

micro-os-plus-iii.git/
├─ CMakeLists.txt           # the library under test: micro-os-plus::iii
├─ package.json             # xPack metadata (name, version, tooling)
├─ .github/workflows/ci.yml # GitHub Actions: matrix of OSes
└─ tests/                  # ← everything test-related lives here
    ├─ package.json         # the xPack "tests" package: the matrix
    ├─ CMakeLists.txt       # top-level test build entry point
    ├─ cmake/
    │   ├─ tests-main.cmake # common build flow (generated)
    │   ├─ common-options.cmake # warnings, C11/C++20, common flags
    │   └─ global-definitions.cmake # ENABLE_*_TEST switches
    ├─ sources/             # platform-agnostic test sources
    │   ├─ rtos-apis/       # test::rtos-apis
    │   ├─ mutex-stress/    # test::mutex-stress
    │   ├─ cmsis-os-validator/ # test::cmsis-os-validator
    │   ├─ instrumentation/ # (instrumentation support)
    │   └─ blinky/          # (simple demo/led test)
    ├─ platforms/           # one folder per supported platform
    │   ├─ native/          # host process (gcc / clang)
    │   ├─ qemu-cortex-m0/  # QEMU mps2-an385 (M3 machine, M0 code)
    │   ├─ qemu-cortex-m3/  # QEMU mps2-an385
    │   ├─ qemu-cortex-m4f/ # QEMU mps2-an386
    │   ├─ qemu-cortex-m7f/ # QEMU mps2-an500
    │   ├─ raspberrypi-pico/ # RP2040, Cortex-M0+, OpenOCD/CMSIS-DAP
    │   ├─ nucleo-f411re/   # STM32F411, Cortex-M4F, ST-Link
    │   ├─ nucleo-f767zi/   # STM32F767, Cortex-M7F, ST-Link
    │   └─ nucleo-h743zi/   # STM32H743, Cortex-M7F, ST-Link
    ├─ device-qemu-cortexm/ # device package for the QEMU boards
    ├─ xpacks/              # installed/linked dependencies (tools)
    ├─ build/               # per-configuration build trees (ignored)
    └─ deprecated/         # legacy tests, kept for history

```

A platform folder has a consistent internal shape:

Layout

```

tests/platforms/<platform>/
├─ CMakeLists.txt           # defines the test executables + add_test()
├─ README.md                # memory map, invocation, prerequisites
├─ include/                 # platform include headers
├─ cmake/
    ├─ definitions.cmake    # PLATFORM / DEVICE / linker-script names
    ├─ dependencies-folders.cmake # which subdirs to add_subdirectory()
    └─ platform-library.cmake # the micro-os-plus::platform interface lib
└─ device/ or stm32*cubemx/ # device support (STM32 variants)

```

5. The xPack / xpm layer

5.1 What xpm and xPacks are

xpm is a portable Node.js CLI (`xpm`, install via `npm install -g xpm`) that adds C/C++-oriented features on top of npm:

- installs binary **tool xPacks** (@xpack-dev-tools/...) into xpacks/ instead of node_modules/, with per-platform archives;
- supports multiple **build configurations** per project;
- exposes a task runner: `xpm run <action> --config <config>.`

A project is an ordinary npm package with an extra "xpack" section in `package.json`. xpm uses that section to know the required minimum version, the devDependencies to install, the available build configurations, and the named actions.

5.2 The two `package.json` files

File	Role
package.json (root)	The library under test. minimumXpmRequired: 0.20.8, engines.node >= 20, dev dependency @xpack-dev-tools/clang.
tests/package.json	The test harness. Declares the tool xPacks and the whole build-configuration matrix. This is the file to edit to change coverage.

The tests package pins the *orchestration* tools at the top level (tests/package.json:27-45):

File: micro-os-plus-iii.git/tests/package.json

```
// micro-os-plus-iii.git/tests/package.json
"devDependencies": {
  "@xpack-dev-tools/cmake": "3.26.5-1.1",
  "@xpack-dev-tools/ninja-build": "1.11.1-3.1",
  "@micro-os-plus/build-helper": { "specifier": "^2.17.0", "local": "link" },
  "@xpacks/arm-cmsis-rtos-validator": { ... },
  "@xpacks/chan-fatfs": { ... }
}
```

The *compiler / emulator / debugger* tools are pinned per configuration, so a `gcc11` build and a `clang19` build can coexist with different tool versions.

5.3 tests/xpacks/ and .bin

After `xpm install` (and one `xpm install --config <name>` per configuration), the tree contains:

Layout

```

tests/xpacks/
├─ .bin/                # shims for the top-level tools (cmake, ninja, ctest...)
├─ @xpack-dev-tools/    # top-level binary tools
├─ @micro-os-plus/build-helper -> ~/.local/xPacks/... # linked dev repo
└─ @xpacks/             # source xPacks (validator, chan-fatfs)

tests/build/<config>/xpacks/
├─ .bin/                # per-config tool shims (arm-none-eabi-gcc, qemu-system-arm...)
└─ @xpack-dev-tools/    # per-config binary tools

```

`.bin` is prepended to `PATH` by `xpm` when it runs an action, which is how the CMake build finds `qemu-system-arm`, `openocd`, `arm-none-eabi-gcc`, etc. The CMake test commands deliberately call these by **bare name** (`qemu-system-arm`, `openocd`, `arm-none-eabi-*`), relying on the `xpm`-provided `PATH` — see `tests/platforms/qemu-cortex-m7f/CMakeLists.txt:64` and `tests/platforms/nucleo-f767zi/CMakeLists.txt:69`.

5.4 local: "link" and development

Dependencies declared with `"local": "link"` are resolved by `xpm link @scope/name`, which symlinks the local development clone into `tests/xpacks/` instead of copying a published package. This is what makes it possible to iterate on `micro-os-plus-iii-cortexm`, `build-helper`, `arm-cmsis`, `chan-fatfs`, `libucontext`, the Pico SDK, etc., and see the changes immediately. The `git-clone-deps` action (`tests/package.json:59-84`) clones and links all of them in one shot.

6. The build-configuration matrix

`tests/package.json` → `xpack.buildConfigurations` is the heart of the system. Each entry is a named configuration. The naming convention is:

Layout

```

<platform>-cmake-<toolchain>-<buildtype>
  native-cmake-gcc14-debug
  native-cmake-clang19-release
  qemu-cortex-m7f-cmake-gcc-debug
  raspberrypi-pico-cmake-gcc-release

```

6.1 Hidden building blocks

Instead of repeating flags in every configuration, the file defines **hidden** fragments that configurations `inherit` from:

Hidden block	Purpose
<code>cmake-actions</code>	prepare, build, test, clean → the standard CMake/CTest lifecycle
<code>native-actions</code>	install + link-deps for host builds (links <code>posix-arch</code> , <code>libucontext</code>)

Hidden block	Purpose
cortexm-actions	install + link-deps for embedded builds (links iii-cortexm, arm-cmsis)
native-dependencies	@micro-os-plus/posix-arch, @xpack-3rd-party/libucontext
cortexm-dependencies	@micro-os-plus/micro-os-plus-iii-cortexm
arm-cmsis-dependencies	@xpacks/arm-cmsis
arm-none-eabi-gcc-dependencies	@xpack-dev-tools/arm-none-eabi-gcc
gccNN-dependencies / clangNN-dependencies	pinned host compilers
qemu-arm-dependencies	@xpack-dev-tools/qemu-arm
openocd-dependencies	@xpack-dev-tools/openocd
short-win-paths-properties	short build folder names on Windows (m7fd, nf7r, ...)

Example (tests/package.json:958-974), abbreviated:

File: [micro-os-plus-iii.git/tests/package.json](https://github.com/micro-os-plus-iii.git/blob/master/tests/package.json)

```
// micro-os-plus-iii.git/tests/package.json
"qemu-cortex-m0-cmake-gcc-debug": {
  "inherit": [
    "cortexm-actions", "cmake-actions", "cortexm-dependencies",
    "arm-cmsis-dependencies", "arm-none-eabi-gcc-dependencies",
    "qemu-arm-dependencies", "short-win-paths-properties"
  ],
  "properties": {
    "buildType": "Debug",
    "platformName": "qemu-cortex-m0",
    "toolchainFileName": "arm-none-eabi-gcc.cmake",
    "shortConfigurationName": "m0d"
  }
}
```

A release configuration simply inherits its debug sibling and overrides buildType (to Release for native, MinSizeRel for Cortex-M).

6.2 Properties used by the actions

The templated command strings use these properties (Liquid syntax):

Property	Meaning
buildFolderRelativePath	build/<configuration.name> (lowercased)
buildFolderRelativePathPosix	same, with forward slashes
buildType	CMake CMAKE_BUILD_TYPE (Debug/Release/MinSizeRel)

Property	Meaning
platformName	passed to CMake as <code>-D PLATFORM_NAME=...</code>
toolchainFileName	file in <code>build-helper/cmake/toolchains/</code>
shortConfigurationName	short folder name on Windows
commandCMakeReconfigure	<code>cmake -S . -B <build> -G Ninja -D CMAKE_BUILD_TYPE=... -D PLATFORM_NAME=...</code>
commandCMakePrepareWithToolchain	<code>reconfigure + -D CMAKE_TOOLCHAIN_FILE=xpacks/@micro-os-plus/build-helper/cmake/toolchains/<file></code>
commandCMakeBuild	<code>cmake --build <build></code>
commandCMakePerformTests	<code>cd <build> && ctest -V</code>

These are defined in `tests/package.json:46-56`.

6.3 The four standard actions

Every `cmake-actions` configuration exposes:

Action	What it does
prepare	configure CMake, including the toolchain file
build	reconfigure + <code>cmake --build</code> (Ninja)
test	<code>cd build/<config> && ctest -V</code>
clean	<code>cmake --build <build> --target clean</code>

And `native-actions` / `cortexm-actions` add:

Action	What it does
install	<code>xpm install --config <name></code> (fetches the pinned tools)
link-deps	<code>xpm link ...</code> the development packages into the config

6.4 Aggregate actions (the entry points)

`tests/package.json` also defines high-level actions that fan out over many configurations:

Action	Coverage
test	default: <code>test-qemu-cortex-m7f-cmake</code>
install-all	<code>npm install + xpm install --all-configs</code>
test-all	<code>test-native-cmake + test-cortex-cmake</code>
test-native-cmake	host GCC 11-14 + clang 16-19, debug & release (on macOS: system compiler + clang 16-19; clang 13-15 currently disabled)
test-cortex-cmake	QEMU M0, M3, M4F, M7F, debug & release
install-ci / test-ci	the CI subset (OS-aware via Liquid <code>{% if os.platform ... %}</code>)


```
# micro-os-plus-iii.git/tests/cmake/tests-main.cmake
include("cmake/common-options.cmake")           # micro-os-plus::common-options
include("platforms/${PLATFORM_NAME}/cmake/definitions.cmake")
include("platforms/${PLATFORM_NAME}/cmake/dependencies-folders.cmake")
xpack_add_dependencies_subdirectories("${xpack_dependencies_folders}" "xpacks-bin")
add_subdirectory("../top-bin")                  # the library under test
add_subdirectory("platforms/${PLATFORM_NAME}" "platform-bin") # executables + tests
```

(tests/cmake/tests-main.cmake:41-65.)

7.2 common-options — the shared compile/link interface

tests/cmake/common-options.cmake defines the interface library `micro-os-plus::common-options` (alias of `micro-os-plus-common-options-interface`) which every test links:

- `-fmessage-length=0`, `-fsigned-char`, `-ffunction-sections`, `-fdata-sections`, `-fdiagnostics-color=always`;
- `DEBUG` / `TRACE` only in `Debug`;
- `OS_USE_OS_APP_CONFIG_H` for every build;
- the full warning set from `xpack_set_all_compiler_warnings()` (GCC: `-Wall -Wextra` plus dozens of specific warnings; clang: `-Weverything`).

Most platform libraries add `-Werror` (the Pico SDK is the exception, since it does not allow it), so embedded builds are generally warning-free. See `tests/platforms/qemu-cortex-m7f/cmake/platform-library.cmake:64-65`.

7.3 Platform definitions and dependencies

Each platform provides three CMake fragments:

definitions.cmake — names used by the device package and toolchain:

File: micro-os-plus-iii.git/tests/platforms/<platform>/cmake/definitions.cmake

```
# micro-os-plus-iii.git/tests/platforms/<platform>/cmake/definitions.cmake
set(xpack_device_compile_definition "MICRO_OS_PLUS_DEVICE_QEMU_CORTEX_M7")
set(xpack_platform_compile_definition "MICRO_OS_PLUS_PLATFORM_QEMU_CORTEX_M7F")
set(xpack_device_linker_script_file_name "mem-mps2-an500.ld")
```

(tests/platforms/qemu-cortex-m7f/cmake/definitions.cmake:25-29.)

dependencies-folders.cmake — the list of folders to `add_subdirectory()`. It always includes the test sources, the tested library dependency, the CMSIS package, the portable xPacks, and the device package:

File: micro-os-plus-iii.git/tests/platforms/<platform>/cmake/dependencies-folders.cmake

```
# micro-os-plus-iii.git/tests/platforms/<platform>/cmake/dependencies-folders.cmake
set(xpack_dependencies_folders
  "${CMAKE_SOURCE_DIR}/sources/rtos-apis"
  "${CMAKE_SOURCE_DIR}/sources/mutex-stress"
  "${CMAKE_SOURCE_DIR}/sources/cmsis-os-validator"
  "${CMAKE_BINARY_DIR}/xpacks/@micro-os-plus/micro-os-plus-iii-cortexm"
  "${CMAKE_BINARY_DIR}/xpacks/@xpacks/arm-cmsis"
  "${CMAKE_SOURCE_DIR}/device-qemu-cortexm"
  "${CMAKE_SOURCE_DIR}/xpacks/@xpacks/arm-cmsis-rtos-validator"
  "${CMAKE_SOURCE_DIR}/xpacks/@xpacks/chan-fatfs"
)
```

(tests/platforms/qemu-cortex-m7f/cmake/dependencies-folders.cmake:24-37.)

Note the two roots: `CMAKE_SOURCE_DIR` for source xPacks, and `CMAKE_BINARY_DIR` for the per-config dependencies installed by xpm.

platform-library.cmake — creates the interface library `micro-os-plus::platform` (alias of `platform-<name>-interface`). This is where CPU flags, float ABI, `-nostartfiles`, `--gc-sections`, linker script selection, RPATH handling (native), and the `platform`→`library` links live. For Cortex-M platforms it links `micro-os-plus::iii-cortexm` and `micro-os-plus::device` (tests/platforms/qemu-cortex-m7f/cmake/platform-library.cmake:110-113).

7.4 The library under test and the device package

- The **library under test** is the repository root `CMakeLists.txt`, producing the interface library `micro-os-plus::iii` (`CMakeLists.txt:41-140`). It enumerates every source file of the portable $\mu\text{OS++}$ core.
- The **device package** `tests/device-qemu-cortexm` produces `micro-os-plus::device`: system init, vector table, exception handlers and the linker scripts (`tests/device-qemu-cortexm/CMakeLists.txt:25-60`).

For physical STM32 boards the device code lives in the platform folder (`stm32f411cubemx/`, `stm32f767cubemx/`, `stm32h743cubemx/`), added via `add_subdirectory()` from the platform `CMakeLists.txt`.

7.5 Test sources as interface libraries

Each test in `tests/sources/<name>/CMakeLists.txt` defines an interface library and a namespaced alias:

Test	Target	Alias
RTOS C & C++ APIs, FatFS	test-rtos-apis-interface	test::rtos-apis
Mutex stress & uniformity	test-mutex-stress-interface	test::mutex-stress
Arm CMSIS OS validator	test-cmsis-os-validator-interface	test::cmsis-os-validator

They list their `INTERFACE` sources (so they are compiled into whichever executable links them) and expose their `include/` folder. Example: `tests/sources/rtos-apis/CMakeLists.txt:30-45`.

7.6 Per-platform executables and CTest registration

The platform `CMakeLists.txt` is where the two halves meet. It defines an `add_test_executable()` helper and, for each enabled test, an executable that links:

Libraries linked

```
micro-os-plus::common-options    # flags/warnings
test::<name>                      # the test sources
micro-os-plus::iii                # the library under test
[portable deps]                  # e.g. xpacks::chan-fatfs
micro-os-plus::platform          # platform glue
```

then registers it with CTest:

- **native** (`tests/platforms/native/CMakeLists.txt:75`): `add_test(NAME "rtos-apis-test" COMMAND rtos-apis-test)`
- **QEMU** (`tests/platforms/qemu-cortex-m7f/CMakeLists.txt:61-68`):

```
add_test(NAME "rtos-apis-test" COMMAND
  qemu-system-arm --machine mps2-an500 --cpu cortex-m7 --kernel rtos-apis-test.elf
  --nographic -d unimp,guest_errors
  --semihosting-config enable=on,target=native,arg=rtos-apis-test)
```

- **physical board via OpenOCD** (`tests/platforms/nucleo-f767zi/CMakeLists.txt:66-73`):

```
add_test(NAME "rtos-apis-test" COMMAND
  openocd -c "gdb port disabled" -c "tcl port disabled" -c "telnet port disabled"
  -f interface/stlink-dap.cfg -f target/stm32f7x.cfg
  -c "program rtos-apis-test.elf verify" -c "arm semihosting enable"
  -c "arm semihosting_cmdline rtos-apis-test" -c "reset")
```

This is the single most important design point: **CTest does not care whether the “program under test” is a host process, an emulator or a debug probe.** It runs a command and checks the exit status. Because semihosting forwards the target’s `stdout`/exit code to the host, a QEMU or OpenOCD run behaves exactly like a native run for CTest purposes.

7.7 Build-helper CMake functions

The shared xPack `@micro-os-plus/build-helper` (`~/local/xPacks/@micro-os-plus/build-helper/<ver>/`) provides:

- `xpack_add_dependencies_subdirectories()` — iterate a folder list and `add_subdirectory()` each, failing loudly if `CMakeLists.txt` is missing (`@micro-os-plus/build-helper/cmake/micro-os-plus-build-helper.cmake:100-116`);
- `xpack_set_all_compiler_warnings()` — the exhaustive warning set;
- `xpack_display_target_lists()` / `xpack_display_greetings()` — verbose diagnostics, enabled with `--log-level=VERBOSE`;
- `xpack_add_cross_custom_commands()` — post-build `size` (and optional `objcopy/objdump`) steps for cross builds;

- `xpack_get_package_name_and_version()` — reads `package.json` into CMake;
- toolchain files under `cmake/toolchains/`: `gcc.cmake`, `clang.cmake`, `arm-none-eabi-gcc.cmake`, `aarch64-none-elf-gcc.cmake`, `riscv-none-elf-gcc.cmake`, `riscv-none-embed-gcc.cmake`.

Because this is a shared xPack, the same conventions (and future platforms such as RISC-V) are available to every `µOS++` project.

8. Platforms in detail

8.1 Native

- **Toolchains:** host GCC 11/12/13/14, clang 13/14/15/16/17/18/19.
- **Runner:** the test is an ordinary host process; `add_test` runs it directly.
- **Dependencies:** `@micro-os-plus/posix-arch` and `@xpack-3rd-party/libucontext` (synthetic POSIX scheduler).
- **Details:** RPATH is computed by asking the compiler for its library paths (`build-helper/dev-scripts/get-libraries-paths.sh`) and passed via `-Wl,-rpath` (`tests/platforms/native/cmake/platform-library.cmake:37-61`). On clang it uses `libc++`, `compiler-rt`, `lld`, and `libunwind`. On Windows everything is static.

8.2 QEMU Cortex-M

Platform	QEMU machine	QEMU --cpu	Compile flags	Notes
<code>qemu-cortex-m0</code>	<code>mps2-an385</code>	<code>cortex-m3</code>	<code>-mcpu=cortex-m0</code> <code>-mfloat-abi=soft</code>	M0 code run on an M3 board; no hardware divide
<code>qemu-cortex-m3</code>	<code>mps2-an385</code>	<code>cortex-m3</code>	<code>-mcpu=cortex-m3</code> <code>-mfloat-abi=soft</code>	
<code>qemu-cortex-m4f</code>	<code>mps2-an386</code>	<code>cortex-m4</code>	<code>-mcpu=cortex-m4</code> <code>-mfloat-abi=hard</code>	DSP + FPU
<code>qemu-cortex-m7f</code>	<code>mps2-an500</code>	<code>cortex-m7</code>	<code>-mcpu=cortex-m7</code> <code>-mfloat-abi=hard</code>	double-precision FPU

- **Toolchain:** `arm-none-eabi-gcc 14.2.1`; hard-float only on M4F/M7F.
- **Execution:** fully semihosted. Tests are compiled with `OS_USE_TRACE_SEMIHOSTING_STDOUT` and QEMU is launched with `--semihosting-config enable=on,target=native,arg=<test>`.
- **Device:** `tests/device-qemu-cortexm` supplies the vector table, system init and linker scripts `mem-mps2-an385.ld` / `an386.ld` / `an500.ld` (heap/stack derived from `SEMIHOSTING_SYS_HEAPINFO`).

8.3 Physical boards (OpenOCD semihosting)

Platform	MCU	Core	Probe config
raspberrypi-pico	RP2040	Cortex-M0+	interface/cmsis-dap.cfg + target/rp2040.cfg
nucleo-f411re	STM32F411	Cortex-M4F	interface/stlink-dap.cfg + target/stm32f4x.cfg
nucleo-f767zi	STM32F767	Cortex-M7F	interface/stlink-dap.cfg + target/stm32f7x.cfg
nucleo-h743zi	STM32H743	Cortex-M7F	interface/stlink-dap.cfg + target/stm32h7x.cfg

OpenOCD programs the ELF, enables Arm semihosting, passes the test name as the semihosting command line, and resets the target. The Pico additionally needs the `bs2_default_padded_checksummed.S` boot second-stage binary (`tests/platforms/raspberrypi-pico/cmake/platform-library.cmake:42-45`). These platforms require the physical hardware and are therefore not part of the default CI run.

9. Toolchains and reproducibility

Reproducibility is achieved by **pinning tool versions in `package.json`** and installing them under `tests/xpacks/`, never relying on whatever happens to be on `PATH`.

Tool	xPack	Pinned version
Host GCC 11-14	@xpack-dev-tools/gcc	11.5.0-2.1 ... 14.2.0-2.1
Host clang 13-19	@xpack-dev-tools/clang	13.0.1-1.1 ... 19.1.7-1.1
Arm cross GCC	@xpack-dev-tools/arm-none-eabi-gcc	14.2.1-1.1.1
QEMU Arm	@xpack-dev-tools/qemu-arm	8.2.6-1.1
OpenOCD	@xpack-dev-tools/openocd	0.12.0-6.1
CMake	@xpack-dev-tools/cmake	3.26.5-1.1
Ninja	@xpack-dev-tools/ninja-build	1.11.1-3.1

Other reproducibility features:

- **Pinned standards:** C 11 and C++ 20, extensions off (`tests/CMakeLists.txt:31-36`).
- **Isolated build trees:** one `build/<configuration.name>` per config; in-source builds are rejected (`tests/CMakeLists.txt:46-51`).
- `compile_commands.json` is always exported for IDE indexing (`tests/cmake/tests-main.cmake:17`).

- **deep-clean** removes `build`, `node_modules`, `xpacks` and `package-lock.json` to start from a known state (`tests/package.json:58`).
- **local**: **"link"** for `µOS++` components, so the exact working tree is used rather than a snapshot.

9.1 CI

`.github/workflows/ci.yml` runs on every push (ignoring docs-only paths) on a matrix of:

CI matrix

```
ubuntu-24.04, ubuntu-24.04-arm, macos-15-intel, macos-15, windows-2025
```

Steps:

CI workflow (`micro-os-plus-iii.git/.github/workflows/`)

```
- npm install -g xpm@0.20.8
- npm --prefix tests ci
- xpm run install-ci -C tests
- xpm run test-ci -C tests
```

`install-ci` and `test-ci` are OS-aware: on Linux they run the GCC (latest, i.e. GCC 14) and clang (latest) native suites; on macOS they run the system compiler (`native-cmake-sys`) and clang; and every OS runs the QEMU Cortex-M0/M3/M7F suites (M4F is not in CI). Physical-board tests are excluded (no hardware in CI). A second, manually triggered workflow (`test-all.yml`, referenced in `tests/README.md`) is planned to run the full matrix.

10. The test suites

The suites are enabled by switches in `tests/cmake/global-definitions.cmake`:

File: micro-os-plus-iii.git/tests/CMakeLists.txt

```
# micro-os-plus-iii.git/tests/CMakeLists.txt
set(ENABLE_RTOS_APIS_TEST true)
set(ENABLE_MUTEX_STRESS_TEST true)
set(ENABLE_CMSIS_OS_VALIDATOR_TEST true)
```

Suite	Sources	What it does
rtos-apis	<code>src/main.cpp</code> , <code>test-c-api.c</code> , <code>test-cpp-api.cpp</code> , <code>test-cmsis-os1.cpp</code> , <code>test-iso-api.cpp</code> , <code>test-posix-io-api.cpp</code> , <code>test-chan-fatfs.cpp</code> , <code>test-cpp-mem.cpp</code> , <code>instrumentation-config.cpp</code>	Exercises the C, C++, CMSIS-OS1, ISO and POSIX-I/O RTOS APIs plus a Chan FatFS test. Entry point <code>os_main()</code> .

Suite	Sources	What it does
mutex-stress	src/main.cpp, src/test.cpp	Random locking from multiple threads over ~30 s, then checks the lock distribution for uniformity. Seeded from the wall clock.
cmsis-os-validator	Arm CMSIS Validator + first external IRQ (WDT_IRQHandler)	Runs the Arm CMSIS RTOS validation suite against the μ OS++ CMSIS-OS compatibility layer.

All suites follow the same contract: `os_main()` returns 0 on success, non-zero on failure, and the exit code is what CTest checks (via semihosting on embedded targets). Trace output is routed to `stdout` either through POSIX (`OS_USE_TRACE_POSIX_STDOUT`, native) or semihosting (`OS_USE_TRACE_SEMIHOSTING_STDOUT`, embedded).

Caveat (from tests/T0-CHECK.md): the tests pass as long as there are no trace prints inside the scheduler interrupt handler. Trace prints in the ISR occasionally cause illegal-instruction faults on native macOS and HardFaults on Cortex-M, less often under emulation and almost always on physical boards.

11. Running the tests

Prerequisites: Node.js $\geq$ 20 and a recent `xpm`.

Commands

```

# 0. install the top-level JS deps
npm --prefix tests install

# 1. install the default config's tools and run the default test
xpm run install -C tests
xpm run test -C tests          # = test-qemu-cortex-m7f-cmake

# 2. host-only, no embedded toolchain
xpm run install-native-cmake-sys -C tests
xpm run test-native-cmake-sys -C tests

# 3. full matrix (all toolchains, all QEMU platforms)
xpm run install-all -C tests
xpm run test-all -C tests

# 4. quick QEMU sweep with the latest toolchains
xpm run install-qemu-cortex-latest -C tests
xpm run run-qemu-cortex-latest -C tests

# 5. single configuration, step by step
xpm run prepare --config qemu-cortex-m7f-cmake-gcc-debug -C tests
xpm run build --config qemu-cortex-m7f-cmake-gcc-debug -C tests
xpm run test --config qemu-cortex-m7f-cmake-gcc-debug -C tests

# 6. reset everything
xpm run deep-clean -C tests

```

The QEMU invocations run *forever* reliably, which is useful for stress testing:

CI script

```

set -e
while (true); do xpm run test-cortex-cmake -C tests; done

```

For a physical board, install the board configuration and run its test action, e.g. `xpm run test-nucleo-f767zi-cmake` with the board connected and permissions set for the probe.

12. How to make a test

This section is about **authoring the test itself** — the code that lives in `tests/sources/<name>/`. Section 13 explains how to wire it into the build.

12.1 The test contract

Every test is a small RTOS application. The rules are:

Rule	Detail
Entry point	

Rule	Detail
	<code>int os_main (int argc, char* argv[])</code> — not <code>main</code> . <code>µOS++</code> startup calls it after the scheduler is initialised.
Return value	0 on success; non-zero on failure. On native this is the process exit code; on embedded it is forwarded to the host by semihosting. CTest checks exactly this.
Platform neutrality	The same sources run everywhere. Platform differences are expressed with preprocessor guards (<code>__ARM_EABI__</code> , <code>__APPLE__</code> , <code>__linux__</code>), never with separate source trees.
I/O	Use <code>printf</code> / <code>puts</code> / <code>trace::printf</code> . The platform CMakeLists chooses the backend (POSIX stdout on native, semihosting on targets).
Threads	Create <code>os::rtos::thread</code> objects and <code>join()</code> them before returning.
No trace in the ISR	Never call <code>trace::printf</code> / <code>printf</code> from inside the scheduler interrupt handler — this is the known cause of intermittent failures (<code>tests/T0-CHECK.md</code>).

`argc/argv` arrive from the platform runner: on QEMU and OpenOCD they come from `--semihosting-config ... arg=<test> / arm semihosting_cmdline <test>`; on native they are the process arguments. Tests may use them to tune runtime (e.g. `mutex-stress` accepts a duration in seconds).

12.2 Directory layout of a test

Layout

```

tests/sources/<name>/
├─ CMakeLists.txt           # INTERFACE library + test::<name> alias
├─ README.md                # what the test proves
├─ include/
│   └─ cmsis-plus/
│       └─ os-app-config.h   # per-test RTOS configuration
│       └─ <name>.h          # optional public declarations
└─ src/
    └─ main.cpp              # os_main() and the test logic

```

The `include/` folder is put on the compiler include path by the test's CMakeLists, so `<cmsis-plus/os-app-config.h>` and `<name>.h` resolve here. Because `micro-os-plus::common-options` defines `OS_USE_OS_APP_CONFIG_H`, the `µOS++` headers automatically include that app config, letting each test tune stack sizes, tick frequency, memory pools and trace switches independently.

12.3 The minimal test

`tests/sources/counter/include/cmsis-plus/os-app-config.h` — copy an existing one (`tests/sources/mutex-stress/include/cmsis-plus/os-app-config.h` is a good starting point) and adjust:

File: micro-os-plus-iii.git/tests/sources/<name>/include/cmsis-plus/os-app-config.h

```
// micro-os-plus-iii.git/tests/sources/<name>/include/cmsis-plus/os-app-config.h
#ifndef CMSIS_PLUS_RTOS_OS_APP_CONFIG_H_
#define CMSIS_PLUS_RTOS_OS_APP_CONFIG_H_

#include "cmsis-plus/platform.h"

#define OS_INTEGER_SYSTICK_FREQUENCY_HZ (1000)

#if defined(__ARM_EABI__)
#define OS_INTEGER_RTOS_MAIN_STACK_SIZE_BYTES (4000)
#elif defined(__APPLE__) || defined(__linux__)
#define OS_INCLUDE_LIBUCONTEXT
#endif

#endif /* CMSIS_PLUS_RTOS_OS_APP_CONFIG_H_ */
```

tests/sources/counter/include/counter.h:

File: [micro-os-plus-iii.git/tests/sources/<name>/include/counter.h](#)

```
// micro-os-plus-iii.git/tests/sources/<name>/include/counter.h
#ifndef COUNTER_H_
#define COUNTER_H_

#if defined(__cplusplus)
extern "C"
{
#endif

    int counter_run (void);

#if defined(__cplusplus)
}
#endif

#endif /* COUNTER_H_ */
```

tests/sources/counter/src/main.cpp:

File: [micro-os-plus-iii.git/tests/sources/<name>/src/counter.cpp](#)

```

// micro-os-plus-iii.git/tests/sources/<name>/src/counter.cpp
#include <cmsis-plus/rtos/os.h>
#include <cmsis-plus/diag/trace.h>

#include <stdio>

#include <counter.h>

int
os_main (int argc __attribute__ ((unused)),
         char* argv[] __attribute__ ((unused)))
{
    trace::printf ("counter test\n");
    return counter_run ();
}

```

The test body uses the public RTOS API. A typical worker-thread pattern:

File: [micro-os-plus-iii.git/tests/sources/<name>/src/counter.cpp](https://github.com/micro-os-plus-iii.git/tests/sources/<name>/src/counter.cpp)

```

// micro-os-plus-iii.git/tests/sources/<name>/src/counter.cpp
using namespace os;
using namespace os::rtos;

struct counter_ctx
{
    mutex m;
    unsigned int value;
};

static void*
counter_worker (void* arg)
{
    auto* ctx = static_cast<counter_ctx*> (arg);
    for (unsigned int k = 0; k < 100; ++k)
    {
        ctx->m.lock ();
        ctx->value += 1;
        ctx->m.unlock ();
        sysclock.sleep_for (1);
    }
    return nullptr;
}

int
counter_run (void)
{
    counter_ctx ctx{ {}, 0 };
    int errors = 0;

    thread w1 ("w1", counter_worker, &ctx);
    thread w2 ("w2", counter_worker, &ctx);

    w1.join ();
    w2.join ();

    if (ctx.value != 200)
    {
        trace::printf ("FAIL: expected 200, got %u\n", ctx.value);
        ++errors;
    }

    puts (errors == 0 ? "Done." : "FAILED.");
    return errors;
}

```

12.4 Assertions and reporting

There is no external assertion framework: a test reports success with its **return code** and with text on the console. Use whichever style fits:

- `assert()` / `__builtin_expect` for invariants; a failed `assert` aborts.
- Explicit checks that print a diagnostic and accumulate an error count, then `return` errors; — this lets one run report *all* failures, which is more useful in CI logs.
- For sub-tests (like `rtos-apis' test_cpp_api()`, `test_c_api()`, ...), call each in sequence and stop or aggregate, printing `errno` between them.

Keep the final line machine-friendly (the existing suites print `done / Done.`), and keep messages free of characters that break semihosting.

12.5 Trace and output backends

The test does not choose the backend — the platform executable does, via `target_compile_definitions`:

Platform	Definition	Where
native	<code>OS_USE_TRACE_POSIX_STDOUT</code>	<code>tests/platforms/native/CMakeLists.txt:56-58</code>
QEMU / OpenOCD	<code>OS_USE_TRACE_SEMIHOSTING_STDOUT</code>	<code>tests/platforms/qemu-cortex-m7f/CMakeLists.txt:31</code>

Both make `printf/trace::printf` reach the host: directly on native, through semihosting on targets. Buffered output (`OS_USE_TRACE_POSIX_FWRITE_STDOUT`) is available but noted as occasionally hanging — prefer line output.

12.6 Platform-specific code

Guard differences instead of duplicating sources. `mutex-stress` is the model (`tests/sources/mutex-stress/src/main.cpp:29-66`): `busy_wait()` uses `hrclock` on `__ARM_EABI__` and `gettimeofday()` elsewhere. Keep such blocks small and always provide both branches so every platform compiles.

12.7 Compiling warning-free

Embedded platforms add `-Werror`, and the common warning set is very aggressive (GCC: `-Wall -Wextra` plus dozens of flags; clang: `-Weverything`). When a warning is genuinely unavoidable, disable it **locally** with a push/pop pair, as the existing tests do:

File: micro-os-plus-iii.git/tests/sources/<name>/src/counter.cpp

```
// micro-os-plus-iii.git/tests/sources/<name>/src/counter.cpp
#pragma GCC diagnostic push
#if defined(__clang__)
#pragma clang diagnostic ignored "-Wpadded"
#elif defined(__GNUC__)
#pragma GCC diagnostic ignored "-Wpadded"
#endif

struct my_msg_s { int i; const char* s; };

#pragma GCC diagnostic pop
```

Do not add global suppressions: fixing the warning is preferred.

13. How to add a test

Wiring a new suite into the harness touches four places. Using a test named `counter` as the running example.

13.1 Step 1 — create the source package

Create `tests/sources/counter/` with the files from section 12, including a `CMakeLists.txt` modelled on `tests/sources/mutex-stress/CMakeLists.txt`:

File: micro-os-plus-iii.git/tests/sources/counter/CMakeLists.txt

```

# micro-os-plus-iii.git/tests/sources/counter/CMakeLists.txt
# tests/sources/counter/CMakeLists.txt
cmake_minimum_required (VERSION 3.20)

add_library (test-counter-interface INTERFACE EXCLUDE_FROM_ALL)

target_include_directories (test-counter-interface INTERFACE "include")

target_sources (test-counter-interface INTERFACE src/main.cpp)

target_compile_definitions (test-counter-interface INTERFACE # None.
)

target_compile_options (test-counter-interface INTERFACE # None.
)

target_link_libraries (test-counter-interface INTERFACE # None.
)

if (COMMAND xpack_display_target_lists)
    xpack_display_target_lists (test-counter-interface)
endif ()

add_library (test::counter ALIAS test-counter-interface)

```

The target name **must** be `test-<name>-interface` and the alias `test::<name>`; the platform `CMakeLists.txt` links the alias.

13.2 Step 2 — add an enable switch

Edit `tests/cmake/global-definitions.cmake`:

File: micro-os-plus-iii.git/tests/CMakeLists.txt

```

# micro-os-plus-iii.git/tests/CMakeLists.txt
set (ENABLE_COUNTER_TEST true)

```

Then guard every platform registration with it. This is the global on/off switch; a platform can still opt out by not adding the folder (step 3) or not registering the test (step 4).

13.3 Step 3 — register the sources on each platform

Add the source folder to `xpack_dependencies_folders` in **every** platform that should run the test, e.g. `tests/platforms/qemu-cortex-m7f/cmake/dependencies-folders.cmake`:

File: micro-os-plus-iii.git/tests/platforms/<platform>/cmake/dependencies-folders.cmake

```
# micro-os-plus-iii.git/tests/platforms/<platform>/cmake/dependencies-folders.cmake
set (xpack_dependencies_folders
  "${CMAKE_SOURCE_DIR}/sources/rtos-apis"
  "${CMAKE_SOURCE_DIR}/sources/mutex-stress"
  "${CMAKE_SOURCE_DIR}/sources/cmsis-os-validator"
  "${CMAKE_SOURCE_DIR}/sources/counter"          # ← new
  ... )
```

If the folder is missing from a platform's list, `test::counter` is undefined and that platform fails to configure. Add it everywhere, or deliberately exclude platforms where the test cannot run.

13.4 Step 4 — create the executable and register the CTest

In each platform's `CMakeLists.txt`, add a guarded block using the platform's `add_test_executable()` helper and its runner command.

Native (`tests/platforms/native/CMakeLists.txt`):

File: `micro-os-plus-iii.git/tests/platforms/<platform>/CMakeLists.txt`

```
# micro-os-plus-iii.git/tests/platforms/<platform>/CMakeLists.txt
if (ENABLE_COUNTER_TEST)
  add_test_executable (counter-test)

  target_compile_definitions (counter-test PRIVATE OS_USE_TRACE_POSIX_STDOUT)

  target_link_libraries (
    counter-test
    PRIVATE micro-os-plus::common-options
            test::counter
            micro-os-plus::iii
            micro-os-plus::platform
  )

  add_test (NAME "counter-test" COMMAND counter-test)
endif ()
```

QEMU (e.g. `tests/platforms/qemu-cortex-m7f/CMakeLists.txt`): the helper already sets `OS_USE_TRACE_SEMIHOSTING_STDOUT` and the cross post-build steps, so only the link and the runner are added:

File: `micro-os-plus-iii.git/tests/platforms/<platform>/CMakeLists.txt`

```
# micro-os-plus-iii.git/tests/platforms/<platform>/CMakeLists.txt
if (ENABLE_COUNTER_TEST)
    add_test_executable (counter-test)

    target_link_libraries (
        counter-test
        PRIVATE micro-os-plus::common-options
                test::counter
                micro-os-plus::iii
                micro-os-plus::platform
    )

    add_test (
        NAME "counter-test"
        COMMAND
            qemu-system-arm${extension} --machine mps2-an500 --cpu cortex-m7 --kernel
            counter-test.elf --nographic -d unimp,guest_errors
            --semihosting-config enable=on,target=native,arg=counter-test
    )
endif ()
```

OpenOCD / physical board (e.g. tests/platforms/nucleo-f767zi/CMakeLists.txt):

File: [micro-os-plus-iii.git/tests/platforms/<platform>/CMakeLists.txt](#)

```
# micro-os-plus-iii.git/tests/platforms/<platform>/CMakeLists.txt
if (ENABLE_COUNTER_TEST)
    add_test_executable (counter-test)

    target_link_libraries (
        counter-test
        PRIVATE micro-os-plus::common-options
                test::counter
                micro-os-plus::iii
                micro-os-plus::platform
    )

    add_test (
        NAME "counter-test"
        COMMAND
            openocd${extension} -c "gdb port disabled" -c "tcl port disabled" -c
            "telnet port disabled" -f interface/stlink-dap.cfg -f target/stm32f7x.cfg
            -c "program counter-test.elf verify" -c "arm semihosting enable" -c
            "arm semihosting_cmdline counter-test" -c "reset"
    )
endif ()
```

If the test needs a portable xPack (as `rtos-apis` needs `xpacks::chan-fatfs`), add it to the `target_link_libraries` list. The executable name convention is `<suite>-test`.

13.5 Step 5 — documentation

- Add the suite to the “Tests details” list in `tests/README.md`.
- Write `tests/sources/counter/README.md` describing what it proves.
- If the test has platform assumptions or known failures, record them in `tests/T0-CHECK.md`.

13.6 No aggregate action needed

Because each configuration’s `test` action runs `ctest -V` in the build tree (`tests/package.json:55`), **every** executable registered with `add_test()` in that platform is picked up automatically. Adding the block from step 4 is enough for `xpm run test-native-cmake-gcc14`, `test-qemu-cortex-m7f-cmake`, `test-all`, `test-ci`, etc. to include the new test. Only add a dedicated `test-counter` action if you want a shortcut that runs just this test.

13.7 Step 6 — build and verify

Commands

```
# fast feedback on the host
xpm run install-native-cmake-sys -C tests
xpm run test-native-cmake-sys -C tests

# embedded
xpm run install-qemu-cortex-latest -C tests
xpm run test-qemu-cortex-m7f-cmake -C tests

# full matrix before pushing
xpm run install-all -C tests
xpm run test-all -C tests
```

To debug a single failure, configure with the toolchain and run `ctest` directly with a filter:

Commands

```
xpm run prepare --config qemu-cortex-m7f-cmake-gcc-debug -C tests
cd tests/build/qemu-cortex-m7f-cmake-gcc-debug
ctest -V -R counter-test
```

13.8 Add-a-test checklist

- ☐ `tests/sources/<name>/` with `CMakeLists.txt`, `README.md`, `include/`, `src/`.
- ☐ Target `test-<name>-interface` and alias `test::<name>`.
- ☐ `include/cmsis-plus/os-app-config.h` present and sane.
- ☐ `ENABLE_<NAME>_TEST` added to `global-definitions.cmake`.
- ☐ Folder added to `dependencies-folders.cmake` on **every** target platform.



Executable + `add_test()` block added to every target platform's `CMakeLists.txt`, with the right trace definition.



`os_main()` returns a meaningful exit code; failure paths print a diagnostic.



Compiles warning-free under GCC and clang (`-Werror` on embedded).



No trace prints inside the scheduler ISR.



`xpm run deep-clean && xpm run install-all && xpm run test-all` passes.

14. Extending platforms, toolchains and coverage

14.1 Add a new platform

1. Create `tests/platforms/<name>/` with the standard layout.
2. Add `cmake/definitions.cmake` (`xpack_platform_compile_definition`, and for embedded targets `xpack_device_compile_definition` and `xpack_device_linker_script_file_name`).
3. Add `cmake/dependencies-folders.cmake` with the full dependency list for that platform.
4. Add `cmake/platform-library.cmake` defining `platform-<name>-interface` and the `micro-os-plus::platform` alias, with the CPU/ABI flags, include dirs, sources and link libraries.
5. Add `CMakeLists.txt` with `add_test_executable()` and the `add_test()` commands for the supported suites.
6. Add a `README.md` documenting the memory map and the invocation.
7. Add one or two build configurations to `xpm.buildConfigurations` in `tests/package.json` (debug + release), inheriting the appropriate hidden blocks and setting `platformName`, `toolchainFileName`, `buildType` and `shortConfigurationName`.
8. Add matching `install-*` / `test-*` / `clean-*` aggregate actions.
9. Add the platform to `tests/README.md` and, if applicable, to CI.

14.2 Add a new toolchain or version

1. Add a hidden `gccNN-dependencies` / `clangNN-dependencies` block (or a new tool `xPack`) pinning the version.
2. Add a pair of build configurations inheriting it, with the right `toolchainFileName`.
3. Wire them into the aggregate `test-native-cmake-*` actions.
4. Optionally add a toolchain file to `build-helper` for a new architecture.

14.3 Enable / disable coverage

- Flip the `ENABLE_*_TEST` variables in `global-definitions.cmake` (global) or guard an `add_test_executable()` call (per platform).
- Add or remove entries in the aggregate actions to change what `test-all` covers.
- `-D PLATFORM_NAME=...` is what selects the platform tree; adding a config with a new `platformName` is all that is needed to build it.

14.4 Extension checklist

- ☐ New sources compile warning-free (`-Werror` on embedded).
 - ☐ `os_main()` returns a meaningful exit code.
 - ☐ Trace output uses the platform's trace backend (POSIX or semihosting).
 - ☐ No trace prints inside the scheduler ISR.
 - ☐ `dependencies-folders.cmake` updated on **every** platform that should run the suite.
 - ☐ Configurations added for both debug and release.
 - ☐ `README.md` and aggregate actions updated.
 - ☐ `xpm run deep-clean && xpm run install-all && xpm run test-all` passes.
-

15. Chaining and inheritance rules (`package.json` and `CMakeLists.txt`)

The harness is held together by several distinct **chains**. A chain is a defined order in which one file contributes to the next. Knowing the rules of each chain is what makes changes predictable.

There are five chains:

1. **xpm configuration inheritance** — `buildConfigurations` inherit from each other (`inherits`).
2. **xpm property/template chaining** — properties and actions reference other properties through `Liquid`.
3. **xpm action chaining** — aggregate actions call per-configuration actions, which call the standard lifecycle actions.
4. **CMake include / `add_subdirectory` chaining** — a fixed configure-time order.
5. **CMake target (usage-requirement) chaining** — interface libraries propagate flags, sources and link libraries transitively.

15.1 `package.json` — configuration inheritance rules

A configuration may list parents in `inherits` (the older spelling `inherit` is deprecated but still accepted; this repository currently uses `inherit`). The rules are:

Rule	Behaviour
Recursive	Each parent is fully initialised (including <i>its</i> parents) before the child merges from it.
Ordered	Parents are merged in the order listed: later entries override earlier ones.

Rule	Behaviour
Local wins	The configuration's own <code>properties</code> , <code>dependencies</code> , <code>devDependencies</code> and <code>actions</code> are merged last and override everything inherited.
Properties	Merged key by key (later parent wins, local wins over all).
Dependencies / devDependencies	Merged key by key, same precedence.
Actions	Merged by action name; a local action with the same name replaces the inherited one.
Cycles	Circular inheritance is detected and rejected with an error.
Hidden	<code>"hidden": true</code> marks a building block: it is fully initialised and inheritable, but skipped by <code>--all-configs</code> , not meant to be selected directly, and does not get a build folder.

This is why the repo can express an entire matrix with a handful of mixins:

File: micro-os-plus-iii.git/tests/package.json

```
// micro-os-plus-iii.git/tests/package.json
// release inherits debug and only overrides the build type
"qemu-cortex-m0-cmake-gcc-release": {
  "inherit": [ "qemu-cortex-m0-cmake-gcc-debug" ],
  "properties": { "buildType": "MinSizeRel", "shortConfigurationName": "m0r" }
},

// debug inherits several mixins; local properties win over the mixins
"qemu-cortex-m0-cmake-gcc-debug": {
  "inherit": [
    "cortexm-actions", "cmake-actions", "cortexm-dependencies",
    "arm-cmsis-dependencies", "arm-none-eabi-gcc-dependencies",
    "qemu-arm-dependencies", "short-win-paths-properties"
  ],
  "properties": {
    "buildType": "Debug", "platformName": "qemu-cortex-m0",
    "toolchainFileName": "arm-none-eabi-gcc.cmake", "shortConfigurationName": "m0d"
  }
}
```

Practical consequences:

- Changing a **hidden mixin** (`cmake-actions`, `cortexm-actions`, `gccNN-dependencies`, ...) changes **every** configuration that inherits it.
- Changing a **concrete** configuration affects only that configuration (and any configuration that inherits *it*).
- A hidden configuration is skipped by `--all-configs` and is not meant to be selected directly (it has no build folder and usually no platform settings).

15.2 package.json — property / template chaining rules

Properties are evaluated **before** actions, and a property may reference another property. In this repo the chain is explicit:

Properties (package.json)

```
commandCMakePrepare          = "{{ commandCMakeReconfigure }}" --log-level=VERBOSE"
commandCMakePrepareWithToolchain
                              = "{{ commandCMakePrepare }}" -D CMAKE_TOOLCHAIN_FILE=..."
```

Rules and available namespaces:

Namespace	Examples	Notes
properties.*	properties.commandCMakeReconfigure, properties.buildFolderRelativePath	Merged through the inheritance chain; resolved before use.
configuration.*	{{ configuration.name }}	The current concrete configuration.
matrix.*	{{ matrix.arch }}	Only for template configurations.
os.*	{% if os.platform == 'linux' %}	Platform detection for conditional action entries.
env.*, path.*, package.*	environment, path filters, package metadata	Base context.

Additional rules:

- **Build folder.** If `properties.buildFolderRelativePath` is defined it is used; otherwise the default is `build/<sanitized configuration name>`. This repo overrides it only on Windows via `short-win-paths-properties`.
- **Sanitisation.** `{{ configuration.name | to_filename | lowercase }}` produces the folder name, so configuration names and folder names are coupled.
- **Keep property references acyclic.** Because properties may reference other properties, a self-referential chain will not resolve; keep the chain acyclic.
- **The only bridge from xpm to CMake** is the set of `-D` options built from these properties (`PLATFORM_NAME`, `CMAKE_BUILD_TYPE`, `CMAKE_TOOLCHAIN_FILE`).

15.3 package.json — action chaining rules

An action is either a **single command string** or an **array of command strings**. The rules are:

Rule	Behaviour
Array = sequence	Commands in an array run sequentially and stop at the first failure (so <code>set -e</code> -like semantics apply).
Actions call actions	A command may be <code>xpm run <action> --config <config></code> , which is how aggregate actions are built.

Rule	Behaviour
No extra args on arrays	Passing additional CLI arguments to an action whose value is an array is rejected; only a single-command action accepts extra arguments.
Two scopes, not a fallback	With <code>--config X</code> , only X's merged actions (inherited + local) are searched; without <code>--config</code> , only package-level <code>xpack.actions</code> . There is no fallback between the two scopes.
Inherited actions	Actions defined in hidden mixins (e.g. <code>prepare</code> , <code>build</code> , <code>test</code> , <code>clean</code> in <code>cmake-actions</code>) become available in every inheriting configuration.
Substitution context	Per-configuration actions can use <code>properties.*</code> / <code>configuration.*</code> ; package-level actions cannot.

The lifecycle is a three-level chain:

Actions (package.json)

```
test-all
├─ test-cortex-cmake
│   └─ test-qemu-cortex-m7f-cmake
│       ├── prepare --config ...-debug
│       ├── build   --config ...-debug
│       ├── test    --config ...-debug    ← runs `ctest -V` in build/<config>
│       └─ ... release ...
```

Because the leaf `test` action is just `ctest -V`, **every** test registered with `add_test()` is automatically part of every aggregate that reaches it.

15.4 CMakeLists.txt — include / configure chain

CMake processing is strictly ordered. The configure-time chain is:

Layout

```
tests/CMakeLists.txt
├─ enable_testing()                # must precede add_test()
├─ reject in-source builds
├─ list(APPEND CMAKE_MODULE_PATH .../build-helper/cmake)
├─ include("micro-os-plus-build-helper") # defines xpack_* functions
├─ include("cmake/global-definitions.cmake") # ENABLE_*_TEST switches
├─ include("cmake/tests-main.cmake")
│   ├── include("cmake/common-options.cmake") # creates micro-os-plus::common-options
│   ├── include("platforms/${PLATFORM_NAME}/cmake/definitions.cmake")
│   ├── include("platforms/${PLATFORM_NAME}/cmake/dependencies-folders.cmake")
│   ├── xpack_add_dependencies_subdirectories(...) # add_subdirectory each dep
│   ├── add_subdirectory("../" "top-bin")      # micro-os-plus::iii
│   └─ add_subdirectory("platforms/${PLATFORM_NAME}" "platform-bin") # exes + add_test
```

Rules:

Rule	Why it matters
Module path first	<code>include(micro-os-plus-build-helper)</code> must succeed before any <code>xpack_*</code> function is used.
Definitions before consumers	<code>definitions.cmake</code> sets variables consumed by <code>platform-library.cmake</code> and the device package.
Dependencies before users	A dependency's <code>CMakeLists.txt</code> must run before any <code>target_link_libraries()</code> that names its alias.
PLATFORM_NAME selects the tree	It comes from the xpm configuration (<code>-D PLATFORM_NAME=...</code>); there is no hard-coded platform list.
enable_testing() before add_test()	Otherwise the test is not registered with CTest.
Distinct binary dirs	Each <code>add_subdirectory</code> gets its own binary folder (<code>top-bin</code> , <code>platform-bin</code> , <code>xpacks-bin/<name></code>) to avoid collisions.
Generated files are read-only	<code>tests/cmake/tests-main.cmake</code> , <code>common-options.cmake</code> and <code>platform-library.cmake</code> carry DO NOT EDIT! Automatically generated from <code>build-helper/templates</code> . Edit the template in the <code>build-helper</code> xPack, not the copy.

15.5 CMakeLists.txt — add_subdirectory chain and aliases

Each folder listed in `xpack_dependencies_folders` must contain a `CMakeLists.txt` that defines one interface library plus a namespaced alias. The aliases are the linking vocabulary shared across files:

Folder	Target	Alias
repo root	<code>micro-os-plus-iii-interface</code>	<code>micro-os-plus::iii</code>
<code>sources/rtos-apis</code>	<code>test-rtos-apis-interface</code>	<code>test::rtos-apis</code>
<code>sources/mutex-stress</code>	<code>test-mutex-stress-interface</code>	<code>test::mutex-stress</code>
<code>sources/cmsis-os-validator</code>	<code>test-cmsis-os-validator-interface</code>	<code>test::cmsis-os-validator</code>
<code>platforms/<p>/cmake/platform-library.cmake</code>	<code>platform-<p>-interface</code>	<code>micro-os-plus::platform</code>
<code>device-qemu-cortexm</code>	<code>device-qemu-cortexm-interface</code>	<code>micro-os-plus::device</code>
<code>xpacks/.../micro-os-plus-iii-cortexm</code>	—	<code>micro-os-plus::iii-cortexm</code>
<code>xpacks/.../arm-cmsis</code>	—	<code>xpacks::arm-cmsis</code>

Rules:

- **One alias per concept.** `micro-os-plus::platform` is always the alias of the *current* platform library, which is how the test `CMakeLists.txt` stays platform-agnostic.
- **Order follows the list.** `xpack_add_dependencies_subdirectories()` processes `xpack_dependencies_folders` in order and fails loudly if a folder lacks `CMakeLists.txt`.
- **Interface libraries are EXCLUDE_FROM_ALL**, so they compile only as part of an executable that links them.

15.6 CMakeLists.txt — target / usage-requirement chain

CMake `INTERFACE` libraries propagate their requirements transitively. For a test executable the chain is:

Output

```
rtos-apis-test
├─ micro-os-plus::common-options    → C11/C++20 flags, warning set, TRACE/DEBUG
├─ test::rtos-apis                  → test sources + include/
├─ micro-os-plus::iii               → library under test sources + include/
├─ xpacks::chan-fatfs               → portable dependency
└─ micro-os-plus::platform          → platform-<name>-interface
    ├─ CPU / float-ABI / -nostartfiles / --gc-sections / linker script
    ├─ micro-os-plus::iii-cortexm    → Cortex-M port
    └─ micro-os-plus::device         → vectors, startup, exception handlers
        └─ xpacks::arm-cmsis         → CMSIS headers
```

Rules:

- **Flags travel with the library.** The executable never sets CPU flags, warnings or linker options itself; it inherits them by linking. This is why adding a platform is mostly a matter of writing one `platform-library.cmake`.
- **The test library owns only its sources and includes.** Everything else is inherited from the platform and common-options libraries.
- **Adding a dependency = linking its alias** in the platform `CMakeLists.txt`; its usage requirements follow automatically.

15.7 Cross-file contract (who sets what, who consumes it)

Producer	File	Consumed by
<code>platformName</code> property	<code>tests/package.json</code>	<code>-D PLATFORM_NAME</code> → <code>tests-main.cmake</code> selects the platform tree
<code>toolchainFileName</code> property	<code>tests/package.json</code>	<code>-D CMAKE_TOOLCHAIN_FILE</code> → build-helper toolchain
<code>buildType</code> property	<code>tests/package.json</code>	<code>-D CMAKE_BUILD_TYPE</code>
<code>xpack_dependencies_folders</code>	<code>platform dependencies-folders.cmake</code>	<code>tests-main.cmake</code>
<code>xpack_platform_compile_definition</code>	<code>platform definitions.cmake</code>	<code>platform-library.cmake</code>
<code>xpack_device_compile_definition</code>	<code>platform definitions.cmake</code>	<code>device-qemu-cortexm/CMakeLists.txt</code>
<code>xpack_device_linker_script_file_name</code>	<code>platform definitions.cmake</code>	<code>device-qemu-cortexm/CMakeLists.txt</code>
Target aliases (<code>test::...</code> , <code>micro-os-plus::...</code> , <code>xpacks::...</code>)	each <code>CMakeLists.txt</code>	platform executable <code>CMakeLists.txt</code>

Producer	File	Consumed by
Action names (prepare, build, test, clean)	hidden mixins in tests/package.json	aggregate actions
Configuration names	buildConfigurations keys	--config, xpm run ... --config

15.8 Rules of thumb and gotchas

- **Never edit generated CMake files.** Edit the templates in the build-helper xPack, or the per-platform files that are not generated.
- **Order matters** in both inherits (later wins) and xpack_dependencies_folders (targets must exist before they are linked).
- **Prefer inherits** over the deprecated inherit spelling.
- **Hidden configs are mixins only** — they cannot be built.
- **Inheritance cycles are errors**; property self-references will not resolve.
- **Keep the naming conventions** (test::, micro-os-plus::, xpacks::) so aliases resolve consistently everywhere.
- **package.json is the only source of -D values**; CMake never reads package.json except through the variables xpm passes in.
- **A new source folder must be added to every platform that will run it**, otherwise test::<name> is undefined and that platform fails to configure.

16. Known limitations and caveats

- **Trace in the scheduler ISR** occasionally causes illegal instructions on native macOS and HardFaults on Cortex-M (see tests/T0-CHECK.md).
- **Physical-board tests are not in CI** — they need the hardware and a probe.
- **Windows long paths** force short build-folder names (short-win-paths-properties).
- **Pico SDK does not allow -Werror**, so that platform drops it.
- **Some deprecated tests** under tests/deprecated/ have not yet been ported to the new infrastructure.
- The test-all.yml manual workflow referenced in tests/README.md is described as “to be activated soon”.

17. File reference map

Concern	File(s)
Test matrix, tools, actions	tests/package.json
Top-level test build	tests/CMakeLists.txt
Common build flow	tests/cmake/tests-main.cmake
Warnings & common flags	tests/cmake/common-options.cmake
Suite enable switches	tests/cmake/global-definitions.cmake
Library under test	CMakeLists.txt (micro-os-plus::iii)

Concern	File(s)
Test sources	tests/sources/*/CMakeLists.txt
Platform glue	tests/platforms/*/cmake/*.cmake, tests/platforms/*/CMakeLists.txt
QEMU device package	tests/device-qemu-cortexm/CMakeLists.txt
CI	.github/workflows/ci.yml
Shared CMake functions & toolchains	@micro-os-plus/build-helper xPack
Developer/maintainer workflows	README-DEVELOPER.md, README-MAINTAINER.md, tests/README.md
Known failures	tests/T0-CHECK.md

18. Glossary

Term	Meaning
xpm	The xPack package manager CLI (Node.js).
xPack	An npm package with an "xpack" section, used for C/C++ tools and source libraries.
build configuration	A named (platform, toolchain, build type) tuple in package.json.
action	A named command (or list of commands) run with xpm run.
hidden block	A configuration fragment meant only for inherit, not to be built directly.
semihosting	Arm mechanism that lets a target use the host's I/O and exit code; used by QEMU and OpenOCD runs.
CTest	CMake's test runner; ctest -V executes registered add_test commands.
interface library	A CMake target that carries usage requirements (sources, flags, includes) without compiling anything itself.
device package	The xPack providing startup, vectors, linker scripts and exception handlers for a board.
local: "link"	Resolve a dependency to a local development clone instead of a published package.

Generated from an analysis of the `micro-os-plus-iii` `xpack-development` branch. Paths are relative to the repository root unless stated otherwise.